

What is LSTM?

LSTM stands for **Long Short-Term Memory**.

It is a type of **Recurrent Neural Network (RNN)**, which is designed to work with **sequences of data**, like:

- Sentences in chatbots
- Time series data (stock prices, weather)
- Audio or video sequences

Why not a normal neural network?

- Regular neural networks treat each input independently.
- Sentences have **order and context**. For example:
 - "I want a red shirt"
 - "Red I want shirt a"
 - Same words, but meaning is completely different.
- LSTM can **remember previous words** while processing the current word.

How LSTM Works (Simplified)

- LSTM has a **memory cell** that can **remember information for long periods**.
- It has **gates** that control information flow:
 - **Forget Gate:** Decides what information to forget from memory.
 - **Input Gate:** Decides what new information to add to memory.
 - **Output Gate:** Decides what information to output to the next layer.
- This lets LSTM **remember important information** and **ignore irrelevant details** in sequences.

Example in Chatbot

Sentence: "What sizes do you have for sneakers?"

- LSTM reads one word at a time:
 - Reads "What" → starts memory.
 - Reads "sizes" → remembers user wants **size info**.
 - Reads "sneakers" → remembers the product type.
- Output → model predicts **intent**: "size_query"

Without LSTM, the model might **forget “sneakers”** by the time it sees the word "sizes".

☑ Key Points

- LSTM = **special RNN** that remembers long-term dependencies.
- Great for **text, sequences, time-series**.
- Uses **gates** to control memory flow.
- Perfect for chatbots to understand **sentence meaning**.

What Sequential Model Does

A **Sequential model** is like a **pipeline of layers**. Data flows **straight from the first layer to the last layer** without branching.

Think of it like a **conveyor belt**: input → process → output.

2 Step-by-Step Flow

Step 1: Input Layer

- You feed your data into the model.
- Example for a chatbot:
 - Input: "What sizes are available?"
 - After tokenization & padding: [23, 45, 12, 0, 0, ...]
- This becomes the **input to the first layer** (like Embedding).

Step 2: Hidden Layers

- Each layer **processes the data** and passes its output to the next layer.

Example in your model:

1. **Embedding Layer** → Converts word indices → dense vectors $[0.1, 0.2, \dots]$
 2. **First LSTM Layer** → Learns the sequence patterns, remembers important words
 3. **Dropout Layer** → Randomly ignores neurons to prevent overfitting
 4. **Second LSTM Layer** → Learns higher-level sequence features
 5. **Dropout Layer** → Again prevents overfitting
- Each layer transforms the data slightly, **adding “understanding”** at each step.

Step 3: Output Layer

- The last layer produces the final result.
- Example:

```
Dense(len(classes), activation='softmax')
```

- Output: probabilities for each intent.
- Model predicts the intent with **highest probability**.

Step 4: Training (How It Learns)

1. Model makes predictions → compares with **true labels** using a **loss function** (categorical crossentropy).
2. Backpropagation → Adjusts weights in all layers to reduce error.
3. Repeat for **many epochs** until the model learns the patterns.

Step 5: Prediction

- When a user types a message:

- Text is preprocessed → tokenized → padded
- Passed through the Sequential model layer by layer
- Output probabilities → predicted intent
- Bot selects a response

☑ Key Points

- Sequential = **linear stack of layers**
- Each layer only knows about the **previous layer's output**
- Data flows **forward only** (input → hidden layers → output)
- Easy to build LSTM chatbots or simple feedforward networks

Sure! Let's explain **Softmax** clearly and simply.

What is Softmax?

- **Softmax** is an **activation function** used in the **output layer** of a neural network for **multi-class classification**.
- It converts **raw output scores (logits)** into **probabilities** that sum to 1.

Why use Softmax?

- In your chatbot, the model predicts **which intent the user's message belongs to**.
- For example, if there are 5 intents:

```
['greeting', 'size_query', 'color_query', 'brand_query', 'goodbye']
```

- The model's raw outputs might be:

```
[2.0, 1.0, 0.5, 0.3, 0.2]
```

- These are **not probabilities** and don't sum to 1.
- **Softmax converts them** into probabilities:

[0.51, 0.23, 0.14, 0.08, 0.04]

- Now they sum to 1, and the **highest probability (0.51)** corresponds to the predicted intent "greeting".

Softmax Formula

For a vector of outputs [z1, z2, ..., zn]:

[

$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$

]

- $(e^{z_i}) \rightarrow$ exponential of the score
- $(\sum e^{z_j}) \rightarrow$ sum of exponentials of all scores
- Result $\rightarrow$ probability between 0 and 1

In Your Chatbot

`Dense(len(classes), activation='softmax')`

- `len(classes)` $\rightarrow$ number of intents
- Output $\rightarrow$ **probability for each intent**
- Prediction $\rightarrow$ intent with **highest probability**

Key Points

- Softmax = converts scores → probabilities
- Used for **multi-class classification**
- Output probability sum = **1**
- Helps the bot **choose the most likely intent**